thrashing, and optimizes rendering performance—resulting in a faster and more responsive user interface.

This three-phase cycle repeats whenever a new event (such as a user interaction, network response, or state change) triggers a re-render, allowing React to keep the UI in sync with the application's state in an efficient and predictable manner.

Probing question (advanced): "Can you explain how React's Fiber architecture works internally during these phases? Specifically, how does the Fiber tree enable incremental rendering, prioritization, and responsiveness?"

Analysis: This question evaluates the candidate's understanding of React's internal mechanics and its impact on UI responsiveness.

Answer: React's Fiber Architecture, introduced in React 16, fundamentally rewrote its reconciliation engine to support incremental, interruptible rendering. Fiber enables React to break rendering work into small, granular units called fibers, allowing React to pause, resume, or even abort rendering tasks as needed. This capability underpins modern React features like time-slicing and concurrent rendering.

```
At the core of Fiber is the work-in-progress tree, an internal li
{
  tag: WorkTag, #A
  key: string | null, #B
  elementType: any, #C
  pendingProps: any, #D
  memoizedProps: any, #E
  child: Fiber | null, #F
  sibling: Fiber | null, #G
  return: Fiber | null, #H
  stateNode: any, #I
  lanes: number, #J
  ...
};
```

This is how Fiber enables performance optimizations:

- *Incremental rendering (render Phase)*—React traverses the work-in-progress Fiber tree and schedules units of work. Using lanes (priority